Spark performance, with downstream context

An investigation tool that connects execution evidence to the tables and columns a pipeline feeds.

Spark event logs / SQL lineage / In-browser analysis

Why runtime and lineage belong together

The investigation gap

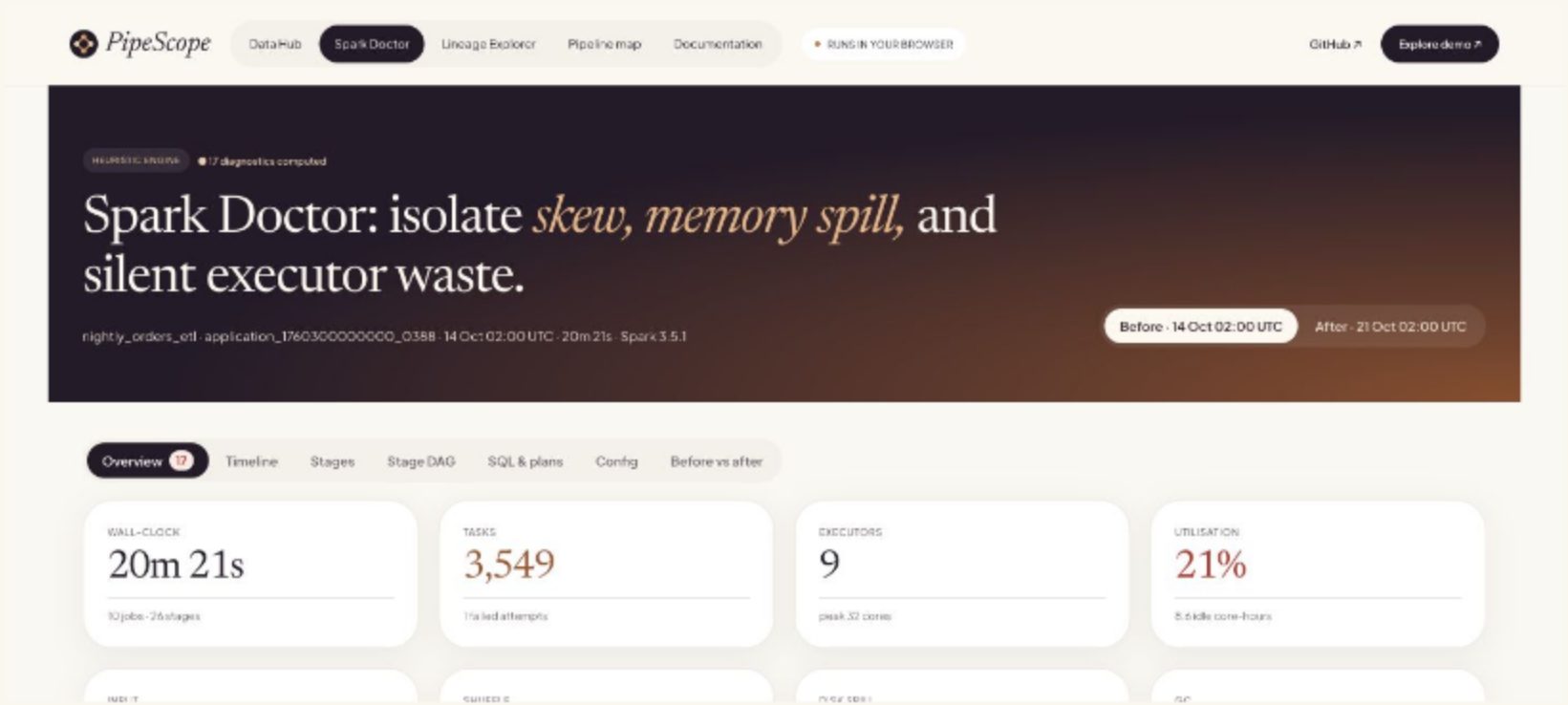
Spark metrics explain where execution slows down. SQL dependencies explain what a change can affect.

My approach

Parse event logs for evidence-backed findings. Resolve SQL at column level. Match statements to the tables they build.

The result is one place to prioritize a bottleneck and inspect its downstream impact.

Diagnostics start with execution evidence



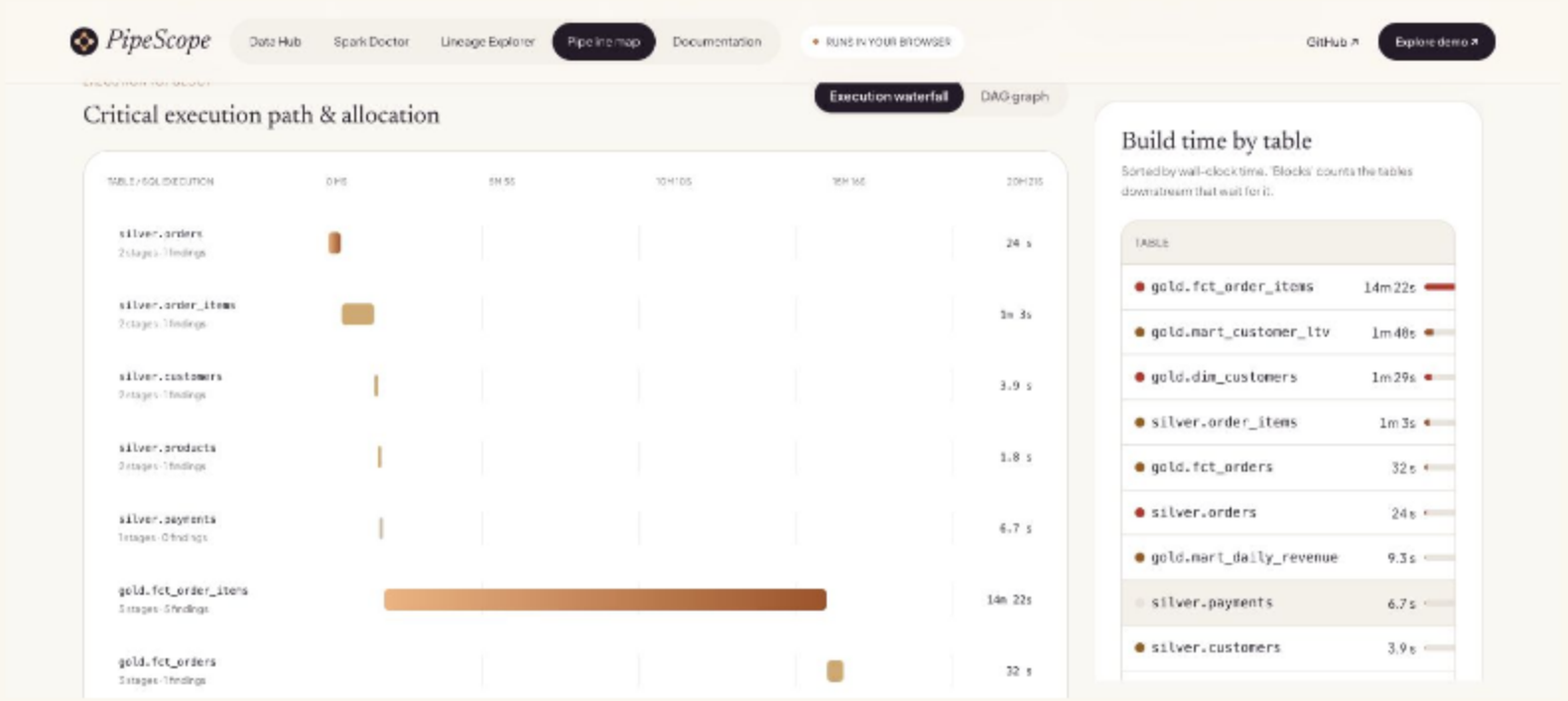
17 rules

Skew, spill, garbage collection, small files and idle cores.

Each finding includes metrics, a threshold and a suggested fix.

Synthetic nightly-orders demo

The slowest build has downstream consequences



14m 22s

Synthetic demo: gold.fct_order_items dominates build time.
The critical path helps distinguish blocking work from parallel work.

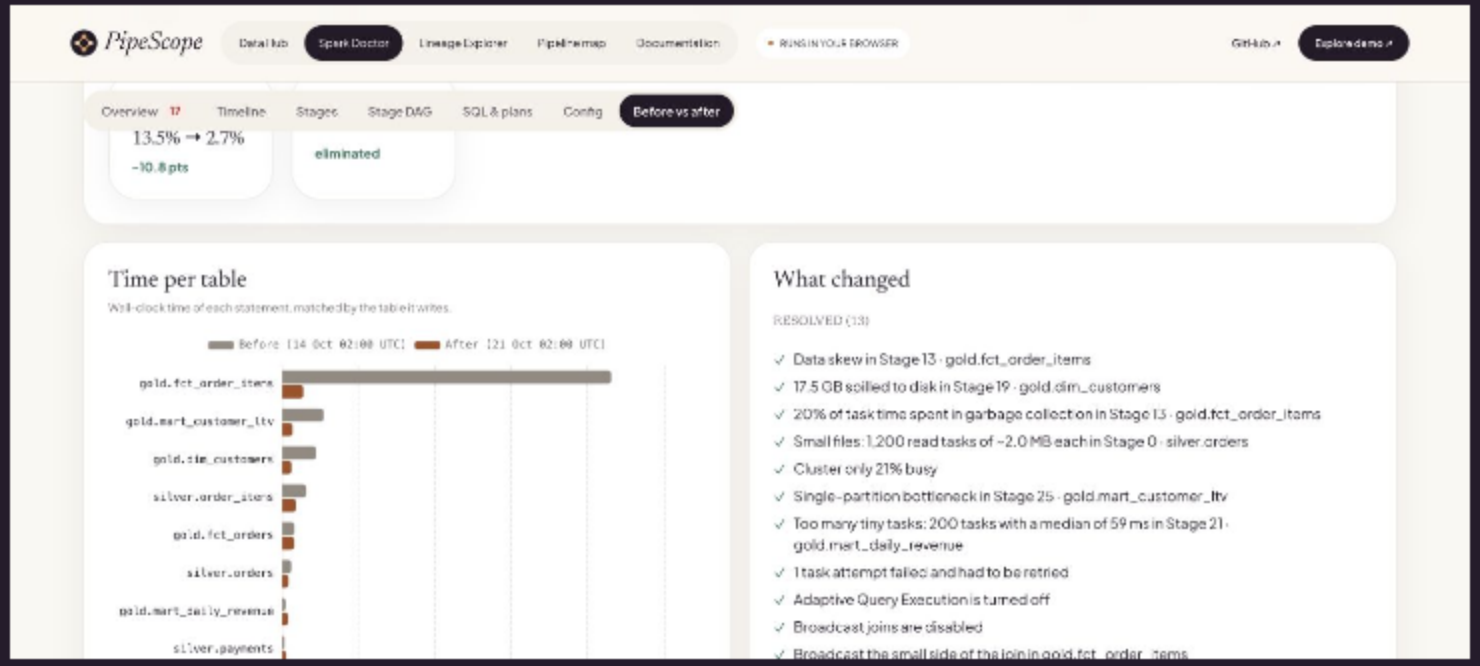
Before-and-after comparison

20m 21s

Before tuning

4m 27s

After tuning



Synthetic demonstration only. The 4.6× ratio illustrates comparison features, not a production result.

How I built the analysis engine

Execution analysis

A Web Worker streams event logs and computes stage percentiles, utilization and diagnostic findings.

SQL dependency analysis

Pyodide runs sqlglot in the browser to resolve column dependencies and SQL or dbt lineage.

Boundaries

SQL lineage does not cover DataFrame code. Gzip is the supported compressed-log format. Diagnostic thresholds need workload context.

adityamalviya04692.github.io/pipescope/